

蓝桥杯30天算法冲刺集训



Day-29 概率与数理统计基础

① Note

本节的知识点内容基本在高中以及本科阶段都已经进行过学习，所以不再赘述。但是如何把学过的理论上的概率论与数理统计，应用到实际上的编程求解问题当中，是本讲义的重点。

蓝桥杯真题

成绩统计（蓝桥杯C/C++2024A组省赛）

可以看到题目让我们求的是方差要小于一个给定的值 T ，我们需要对方差进行一定的了解。

方差定义如下：

方差在统计描述和概率分布中各有不同的定义，并有不同的公式。

在统计描述中，方差用来计算每一个变量（观察值）与总体均数之间的差异。为避免出现离均差总和为零，离均差平方和受样本含量的影响，统计学采用平均离均差平方和来描述变量的变异程度。总体方差公式：

$$\sigma^2 = \frac{\sum (X - \mu)^2}{N}$$

可以发现我们如果要直接计算式子，需要先计算出均值然后再除以个数。这样的话我们要求解本题的话我们要在 $1 \sim x$ 中选出任意 k 名同学，相当于一个 $O(nk)$ 的解法，并且算出来的都是浮点数，可能精度会出现问题。这样的话对于 $1 \leq n, k \leq 10^5$ 来说并不是一个好的解法，因为是会超时的。并且不超时的测试点也有答案错误的可能。

那么本题该如何去求解呢？

首先我们知道数的彼此相差越小，方差越小，那么我们只需要先对一个个前 x 个人排序，然后找一个个的长度为 k 的区间的方差即可，但是这样子去寻找还是没法直接 $O(1)$ 求出长度为 k 的区间的方差，套上二分的话每次check还是会超时。

怎么 $O(1)$ 的求解一个长度为 k 的区间的方差，就是解决本题的关键。

我们可以手推一下这个统计量的式子，我们现在要求的式子如下：

$$\frac{\sum_{i=1}^k (v_i - \bar{v})^2}{k} < T$$

这个式子表示的是对于一个长度为 k 的区间的方差小于 T ，其中 $\bar{v}$ 是代表的均值

我们可以做如下的推导：

$$\begin{aligned}\frac{\sum_{i=1}^k (v_i - \bar{v})^2}{k} &< T \\ \sum_{i=1}^k (v_i - \bar{v})^2 &< kT\end{aligned}$$

我们还可以把不等式左边的统计量展开：

$$\begin{aligned}\sum_{i=1}^k (v_i - \bar{v})^2 &= \sum_{i=1}^k (v_i^2 - 2v_i\bar{v} + \bar{v}^2) \\ &= \sum_{i=1}^k v_i^2 - 2\bar{v} \sum_{i=1}^k v_i + k\bar{v}^2\end{aligned}$$

其中我们知道：

$$\bar{v} = \frac{\sum_{i=1}^k v_i}{k}$$

带入求得

$$\begin{aligned}&\sum_{i=1}^k v_i^2 - 2\bar{v} \sum_{i=1}^k v_i + k\bar{v}^2 \\ &= \sum_{i=1}^k v_i^2 - 2 \frac{(\sum_{i=1}^k v_i)^2}{k} + k \times \frac{(\sum_{i=1}^k v_i)^2}{k^2} \\ &= \sum_{i=1}^k v_i^2 - \frac{(\sum_{i=1}^k v_i)^2}{k}\end{aligned}$$

带回到原来的式子得到：

$$\sum_{i=1}^k v_i^2 - \frac{(\sum_{i=1}^k v_i)^2}{k} < kT$$
$$k \sum_{i=1}^k v_i^2 - (\sum_{i=1}^k v_i)^2 < k^2 T$$

到这一步我们可以看到实际上 $\sum_{i=1}^k v_i^2$ 就是平方的前缀和然后 $(\sum_{i=1}^k v_i)^2$ 就是前缀和的平方，于是我们用前缀和求解即可，这样就能 $O(1)$ 求解出来长度为 k 的区间的方差是不是小于 T 的了。

之后我们还需要注意一下数据范围因为 $1 \leq n, k \leq 10^5$ ， $1 \leq T \leq 2^{31} - 1$ ，所以 $k^2 T$ 也有爆 long long 的可能，所以总体使用 __int128 进行求解。

最终复杂度为 $O(n \log^2 n)$

```
#include <bits/stdc++.h>
using namespace std;
typedef __int128 ll;
const int N = 1e5 + 10;
int aa[N];
ll a[N], pre[N], pre2[N], b[N], n, k, t;
int check(int x)
{
    for(int i=1; i<=x; i++) b[i]=a[i];
    sort(b+1, b+1+x);
    for(int i=1; i<=x; i++)
    {
        pre[i]=pre[i-1]+b[i];
        pre2[i]=pre2[i-1]+b[i]*b[i];
    }
    for(int i=1; i+k-1<=x; i++)
    {
        ll s = k * (pre2[i+k-1] - pre2[i-1]) - (pre[i+k-1] - pre[i-1]) * (pre[i+k-1] - pre[i-1]);
        if(s<k*k*t) return 1;
    }
    return 0;
}
```

```

int main()
{
    int n1,k1,t1;
    cin>>n1>>k1>>t1;
    n=n1,k=k1,t=t1;
    for(int i=1;i<=n;i++) cin>>aa[i],a[i]=aa[i];
    int l=1,r=n1;
    while(r-l>1)
    {
        int mid=(l+r)>>1;
        if(check(mid)) r=mid;
        else l=mid;
    }

    if(check(l)) cout<<l;
    else if(check(r)) cout<<r;
    else cout<<-1;
    return 0;
}

```

故障（蓝桥杯C/C++2022B组国赛）

本题需要使用贝叶斯公式以及全概率公式。这些公式应当在大学本科阶段就已经掌握。

我们假设 $P(A_i)$ 为发生故障原因 i 的概率，这个就是我们输入的 P_i 。假设 $P(B_j)$ 为出现故障现象 j 的概率， $P(B_j|A_i)$ 为在发生故障原因 i 的前提下，出现故障现象 j 的概率，可以看到这个数值就是输入的 P_{ij} 。 $P(A_i|B)$ 为出现当前输入的故障现象的前提下产生故障原因 i 的概率，也可以发现这个数值就是我们最终要求的结果。

那么我们可以根据贝叶斯公式如下，进行求解：

$$P(A_i|B) = \frac{\sum_{j=1}^M P(B_j|A_i)P(A_i)}{P(B)}$$

现在再来解释一下上面的式子，实际上上面的式子由 $P(A|B)$ 而来，但是我们要求的是每个故障原因的概率，所以最终分母还是 $P(B)$ ，分子算的是单独的一个个的故障原因。

而我们目前不知道 $P(B)$ 的概率是多少，我们应当用全概率公式求解 $P(B)$

$$P(B) = \sum_{i=1}^N \sum_{j=1}^M P(B_j|A_i)P(A_i)$$

那么我们带入回原来的式子

$$P(A_i|B) = \frac{\sum_{j=1}^M P(B_j|A_i)P(A_i)}{\sum_{i=1}^N \sum_{j=1}^M P(B_j|A_i)P(A_i)}$$

这样的话我们直接用 $O(NM)$ 时间复杂度遍历概率矩阵求解即可。

但是本题还有一个细节，就是他会给你故障现象出现的编号，也就是并不是所有的故障现象都会发生的。那么我们对于没有发生的故障现象，产生的概率就是 $1 - P_{ij}$ ，其他的都套用上述公式即可。

```
#include<bits/stdc++.h>
using namespace std;
#define N 105
#define int long long
#define For(i, j, n) for(int i = j ; i <= n ; ++i)
int k, n, m, t, vis[N];
long double sum, P[N], p[N][N];
struct node{
    int x;
    long double w;
}a[N];
inline bool cmp(node x, node y){
    if(x.w == y.w) return x.x < y.x;
    return x.w > y.w;
}

signed main(){
    scanf("%lld %lld", &n, &m);
    For(i, 1, n) scanf("%Lf", &P[i]);
    For(i, 1, n) For(j, 1, m) scanf("%Lf", &p[i][j]);
    scanf("%lld", &k);
    For(i, 1, k) scanf("%lld", &t), vis[t] = 1;
    For(i, 1, n){
```

```
    a[i].x = i, a[i].w = P[i];
    For(j, 1, m){
        if(vis[j]) a[i].w *= p[i][j];
        else a[i].w *= (100 - p[i][j]);
    }
    sum += a[i].w;
}
sort(a + 1, a + n + 1, cmp);
For(i, 1, n) printf("%lld %.2Lf\n", a[i].x, a[i].w / sum *
100);
return 0;
}
```